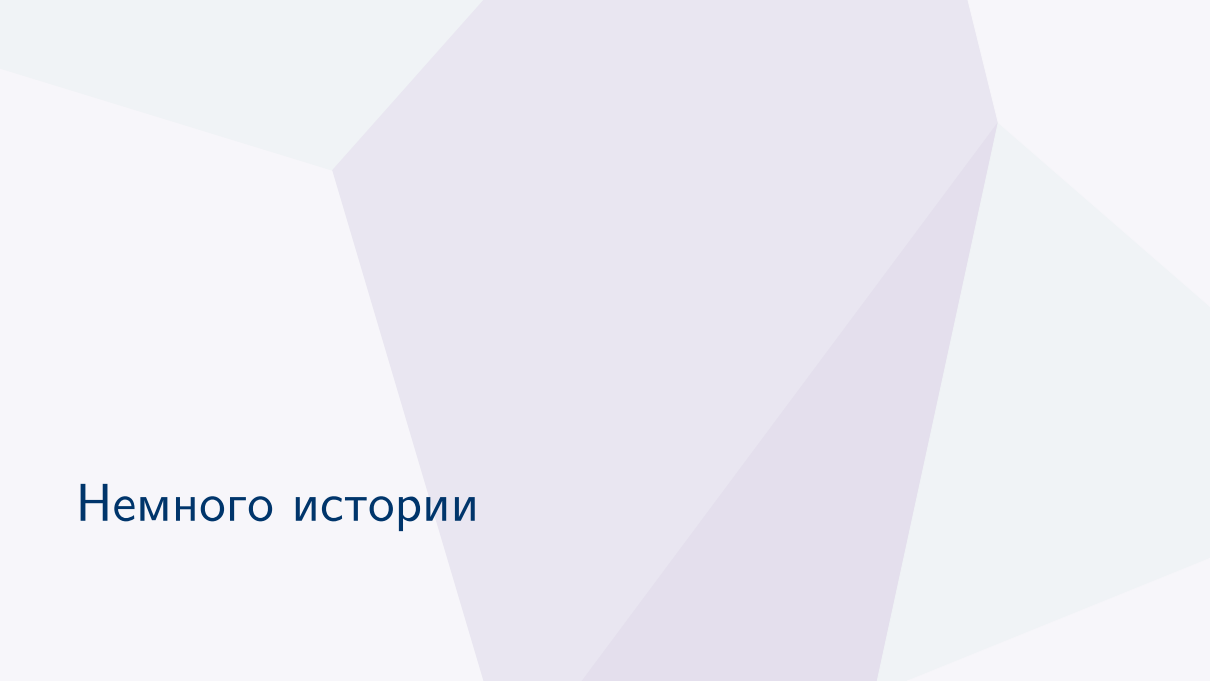


Задание 16. Итоговое повторение

Подготовка к ЕГЭ по информатике



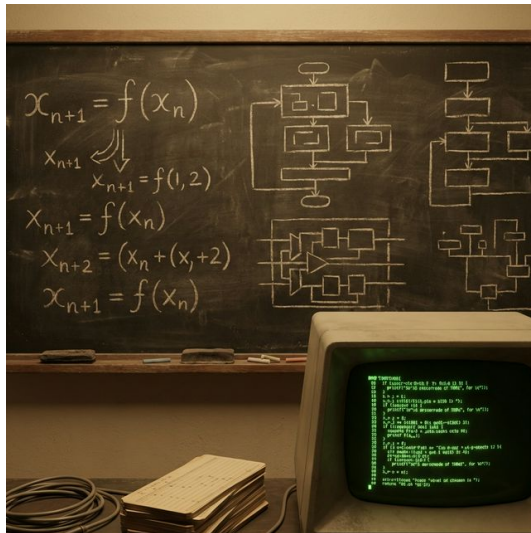
Немного истории

История: Что такое рекурсия?

Рекурсия в программировании и математике — это определение функции (или объекта) через саму себя.

Исторически понятие рекурсии тесно связано с

математической логикой и теорией алгоритмов. Одним из первых, кто формализовал вычислимость через рекурсивные функции в 1930-х годах, был Алонзо Чёрч.



История: Фракталы и самоподобие

В природе и математике рекурсия часто проявляется в виде фракталов — объектов, которые обладают свойством самоподобия (каждая часть подобна целому). Примерами служат снежинка Коха, дерево

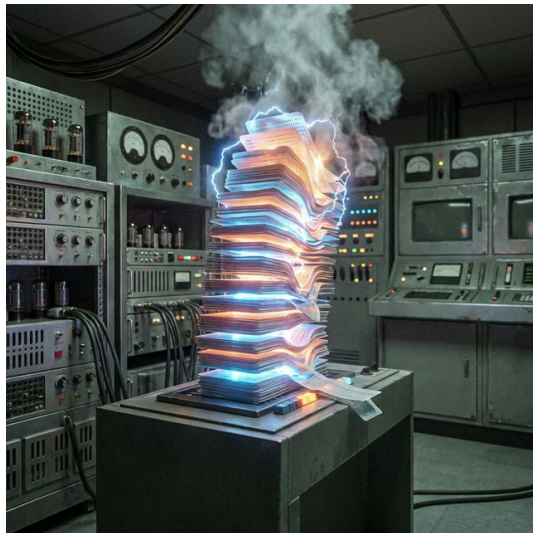
Пифагора, а также природные формы: листья папоротника и кровеносная система человека.



История: Машина Тьюринга и стек вызовов

Когда функция вызывает саму себя, компьютеру необходимо запомнить, куда вернуться после завершения. Для этого используется структура данных под названием **Стек**. При глубокой рекурсии (когда

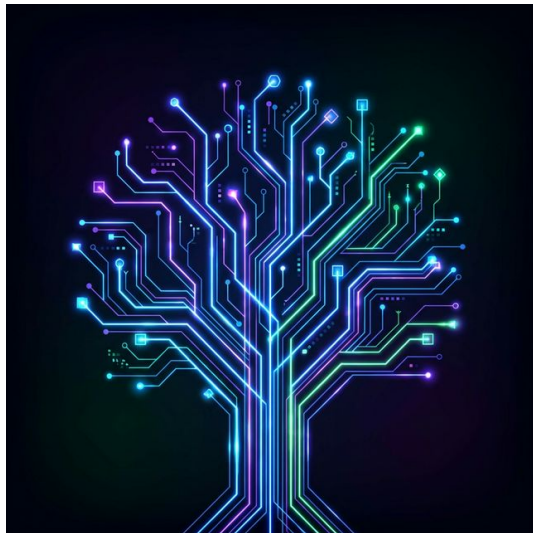
вызовов слишком много) стек переполняется, и происходит ошибка *Stack Overflow*.



История: Дерево вызовов

Для наглядного представления работы рекурсии используют дерево вызовов. В таком дереве корнем является изначальный вызов, а листьями — базовые случаи, которые вычисляются напрямую. Если одна и

та же ветка вычисляется много раз (как в числах Фибоначчи), полезно применять **мемоизацию** (кэширование результатов).





Блиц-опрос

Блиц-опрос: Вопрос 1

Структура рекурсии

Что **обязательно** должно присутствовать в любой рекурсивной функции, чтобы она не работала бесконечно?

- A) Глобальная переменная счётчика.
- B) Базовый случай (условие выхода).
- C) Вызов другой функции.
- D) Кэширование (lru_cache).

Блиц-опрос: Вопрос 2

Стек вызовов

Какая ошибка в Python возникает при превышении лимита рекурсивных вызовов?

- A) MemoryError
- B) IndexError
- C) RecursionError
- D) ValueError

Блиц-опрос: Вопрос 3

Лимиты

Какой модуль и метод в Python используются для увеличения стандартного лимита рекурсии?

- A) `math.setrecursionlimit()`
- B) `sys.setrecursionlimit()`
- C) `os.setrecursionlimit()`
- D) В Python нельзя изменить лимит рекурсии.

Блиц-опрос: Ответы

Правильные ответы

1. **В (Базовый случай)** — без условия выхода (при каком-то n , вычисляемом без рекурсивного вызова), функция будет вызывать саму себя бесконечно.
2. **С (RecursionError)** — ошибка глубины рекурсии (обычно стандартный лимит равен 1000).
3. **В (sys.setrecursionlimit())** — позволяет увеличить глубину стека вызовов, чтобы избежать RecursionError.

Теория: Рекурсивные алгоритмы

Теория: Базовые понятия

Определения

Рекурсия — вызов функцией самой себя.

Базовый случай — условие завершения рекурсии (пример: $F(1) = 1$).

Рекурсивный шаг — переход от текущего состояния к предыдущему (или последующему) с вызовом функции (пример: $F(n) = n \cdot F(n - 1)$).

Теория: Методы решения (Ручной vs Python)

Ручной (Аналитический) метод

Используется для подстановки и сокращения.

$$F(5) = 5 \cdot F(4)$$

$$F(5)/F(4) = (5 \cdot F(4))/F(4) = 5$$

Это спасает, когда значения аргументов ($n > 100000$)

слишком велики даже для `sys.setrecursionlimit`.

Программный метод (Python)

```
import sys sys.setrecursionlimit(5000)
def F(n): if n < 10: return n return n
- 1 + F(n - 1)
print(F(50) - F(48))
```

Теория: Оптимизация (Кэширование)

Зачем нужен кэш?

Если рекурсивная функция вызывает себя несколько раз (числа Фибоначчи), дерево вызовов разрастается в геометрической прогрессии. `lru_cache` запоминает результаты вычислений для конкретных аргументов, ускоряя работу до линейного времени.

Реализация

```
from functools import lru_cache
@lru_cache(None)
def F(n) :
    if n < 3 :
        return 1
    return F(n - 1) + F(n - 2)
```



Практика

Задача 1: Условие

Задача

Алгоритм вычисления функции $F(n)$, где n — целое число, задан следующими соотношениями:

$$F(n) = n, \text{ если } n < 10;$$
$$F(n) = n - 1 + F(n - 1), \text{ если } n \geq 10.$$

Чему равно значение выражения $F(8567) - F(8563)$?

Задача 1: Решение (Аналитически)

Раскрытие рекурсии

Раскроем $F(8567)$, последовательно подставляя рекурсивный шаг, пока не дойдем до $F(8563)$:

$$F(n) = n - 1 + F(n - 1)$$

$$F(8567) = 8566 + F(8566)$$

$$F(8566) = 8565 + F(8565)$$

$$F(8565) = 8564 + F(8564)$$

$$F(8564) = 8563 + F(8563)$$

Подставим всё вместе: $F(8567) = 8566 + 8565 + 8564 + 8563 + F(8563)$

Найти: $F(8567) - F(8563) = 8566 + 8565 + 8564 + 8563 = 34258$.

Ответ

Ответ: 34258

Задача 1: Решение (Python)

```
import sys sys.setrecursionlimit(10000)
def F(n): if n < 10: return n else:
return n - 1 + F(n - 1)
print(F(8567) - F(8563))
```

Анализ кода:

Глубина рекурсии здесь составит около $8567 - 10 = 8557$ вызовов. По умолчанию лимит в Python равен 1000, поэтому нам **обязательно** нужно увеличить его с помощью `sys.setrecursionlimit(10000)`, чтобы программа не выдала `RecursionError`.

Задача 2: Условие

Задача

Алгоритм вычисления значения функции $F(n)$, где n — натуральное число, задан следующими соотношениями:

$$F(n) = n, \text{ при } n > 2024;$$

$$F(n) = n \cdot F(n+1), \text{ если } n \leq 2024.$$

Чему равно значение выражения $F(2022)/F(2024)$?

Задача 2: Решение (Аналитически)

Раскрытие рекурсии

Обратите внимание, что рекурсия идет не вниз $(n - 1)$, а **вверх** $(n + 1)$, пока не достигнет 2024.

Раскроем $F(2022)$:

$$F(2022) = 2022 \cdot F(2023)$$

$$F(2023) = 2023 \cdot F(2024)$$

$$\text{Следовательно: } F(2022) = 2022 \cdot 2023 \cdot F(2024)$$

$$\text{Найти: } \frac{F(2022)}{F(2024)} = \frac{2022 \cdot 2023 \cdot F(2024)}{F(2024)} = 2022 \cdot 2023 = 4090506.$$

Ответ

Ответ: 4090506

Задача 2: Решение (Python)

```
def F(n): if n > 2024: return n else:  
    return n * F(n + 1)  
print(F(2022) / F(2024))
```

Анализ кода:

В данном случае глубина рекурсии очень мала. Нам нужно вычислить $F(2022)$, которое обратится к $F(2023)$, а затем к $F(2024)$. Функция завершится всего за 3 вызова, поэтому изменять лимит рекурсии не нужно, программа отработает мгновенно. Результат деления будет вещественным числом (с `'.0'`), что в ЕГЭ допустимо для анализа.

Задача 3: Условие

Задача

Алгоритм вычисления значения функции $F(n)$, где n — натуральное число, задан следующими соотношениями:

$$F(n) = 1, \text{ если } n = 1;$$

$$F(n) = (n-1) \cdot F(n-1), \text{ если } n > 1.$$

Чему равно значение выражения $(F(2024) + 2 \cdot F(2023))/F(2022)$?

Задача 3: Решение (Аналитически)

Раскрытие рекурсии

Задача содержит сумму выражений в числителе. Раскроем бóльшие инстансы функции до базового $F(2022)$.

$$F(2024) = 2023 \cdot F(2023) = 2023 \cdot 2022 \cdot F(2022)$$

$$F(2023) = 2022 \cdot F(2022)$$

Подставляем в числитель:

$$(2023 \cdot 2022 \cdot F(2022) + 2 \cdot 2022 \cdot F(2022)) / F(2022)$$

Сокращаем $F(2022)$:

$$2023 \cdot 2022 + 2 \cdot 2022 = 2022 \cdot (2023 + 2) = 2022 \cdot 2025 = 4094550.$$

Ответ

Ответ: 4094550

Задача 3: Решение (Python)

```
import sys sys.setrecursionlimit(3000)
def F(n): if n == 1: return 1 else:
return (n - 1) * F(n - 1)
ans = (F(2024) + 2 * F(2023)) //
F(2022) print(ans)
```

Анализ кода:

Так как $n = 2024$, а шаг равен -1 , глубина стека составит 2024, поэтому `sys.setrecursionlimit` необходим. Обратите внимание, что мы используем оператор целочисленного деления `//` для точного результата, так как $F(2024)$ — огромное число (по сути, факториал), и обычное деление `/` может привести к погрешностям с плавающей запятой (*float precision error*) или переполнению.

Задача 4: Условие

Задача

Алгоритм вычисления функции $F(n)$, где n — целое число, задан следующими соотношениями:

$$F(n) = n, \text{ если } n < 10;$$
$$F(n) = (n - 2) \cdot F(n - 5), \text{ если } n \geq 10.$$

Чему равно значение выражения $(F(3220) - 2 \cdot F(3215))/F(3210)$?

В ответе запишите целую часть полученного числа.

Задача 4: Решение (Аналитически)

Раскрытие рекурсии

Шаг функции равен -5 , поэтому $F(3220)$ выражается через $F(3215)$, а то через $F(3210)$:

$$F(3220) = (3220 - 2) \cdot F(3215) = 3218 \cdot F(3215)$$

$$F(3215) = (3215 - 2) \cdot F(3210) = 3213 \cdot F(3210)$$

$$F(3220) = 3218 \cdot 3213 \cdot F(3210) = 10339434 \cdot F(3210)$$

$$\begin{aligned} \text{Выражение: } (F(3220) - 2 \cdot F(3215)) / F(3210) &= \\ &= (10339434 \cdot F(3210) - 2 \cdot 3213 \cdot F(3210)) / F(3210) = \\ &= 10339434 - 6426 = 10333008. \end{aligned}$$

Результат уже является целым числом.

Ответ

Ответ: 10333008

Задача 4: Решение (Python)

```
import sys sys.setrecursionlimit(2000)
def F(n): if n < 10: return n else:
return (n - 2) * F(n - 5)
ans = (F(3220) - 2 * F(3215)) / F(3210)
print(int(ans))
```

Анализ кода:

Глубина рекурсии здесь не такая уж и большая: $(3220 - 10)/5 \approx 642$ вызова. Стандартный лимит в 1000 мог бы справиться, но для надежности мы всё равно его поднимаем. Мы используем `int(ans)`, чтобы взять целую часть числа, как требуется в условии задачи.

Задача 5: Условие

Задача

Алгоритм вычисления функций $F(n)$ и $G(n)$, где n — целое число, задан следующими соотношениями:

$$F(n) = 2 \cdot (G(n-3) + 8);$$

$$G(n) = 2 \cdot n, \text{ если } n < 10;$$

$$G(n) = G(n-2) + 1, \text{ если } n \geq 10.$$

Чему равно значение выражения $F(15548)$?

Задача 5: Решение (Аналитически)

Анализ функции $G(n)$

Сначала выразим F : $F(15548) = 2 \cdot (G(15545) + 8)$. Нужно найти $G(15545)$.

Посмотрим на G . Шаг рекурсии: $n \rightarrow n-2$. Функция прибавляет 1 на каждом шаге. Это арифметическая прогрессия. Из 15545 мы будем спускаться по -2 пока не станем < 10 . Так как 15545 нечетное, мы дойдем до 9.

Количество шагов (прибавлений 1): $k = \frac{15545-9}{2} = 7768$.

То есть $G(15545) = G(9) + 7768$.

Так как $9 < 10$, $G(9) = 2 \cdot 9 = 18$.

Следовательно, $G(15545) = 18 + 7768 = 7786$.

Тогда $F(15548) = 2 \cdot (7786 + 8) = 2 \cdot 7794 = 15588$.

Ответ

Ответ: 15588

Задача 5: Решение (Python)

```
import sys sys.setrecursionlimit(10000)
def G(n): if n < 10: return 2 * n else:
return G(n - 2) + 1
def F(n): return 2 * (G(n - 3) + 8)
print(F(15548))
```

Анализ кода:

В коде программируется две функции: F и G . F не является рекурсивной, она лишь один раз обращается к G . Однако $G(n)$ выполняет около ≈ 7768 вызовов, отсюда возникает необходимость увеличения лимита вызовов (`sys.setrecursionlimit(10000)`).

Задача 6: Условие

Задача

Алгоритм вычисления функций $F(n)$ и $G(n)$, где n — целое число, задан следующими соотношениями:

$$F(n) = 2 \cdot G(n) + G(n - 1);$$

$$G(n) = n, \text{ если } n \leq 10;$$

$$G(n) = G(n-2) + 1, \text{ если } n > 10.$$

Чему равно значение выражения $F(26728)$?

Задача 6: Решение (Аналитически)

Анализ функции $G(n)$

Надо найти $F(26728) = 2 \cdot G(26728) + G(26727)$.

$G(n)$ убавляет по 2 и прибавляет по 1 к счетчику.

Для четного 26728: мы дойдем до 10.

$$G(26728) = G(10) + \frac{26728-10}{2} = 10 + 13359 = 13369.$$

Для нечетного 26727: мы дойдем до 9.

$$G(26727) = G(9) + \frac{26727-9}{2} = 9 + 13359 = 13368.$$

Подставляем в F :

$$F(26728) = 2 \cdot 13369 + 13368 = 26738 + 13368 = 40106.$$

Ответ

Ответ: 40106

Задача 6: Решение (Python)

```
import sys sys.setrecursionlimit(15000)
def G(n): if n <= 10: return n else:
return G(n - 2) + 1
def F(n): return 2 * G(n) + G(n - 1)
print(F(26728))
```

Анализ кода:

Логика полностью идентична предыдущей задаче.

Здесь нам потребуется ≈ 13360 вызовов рекурсии (для вычисления одного $G(n)$), поэтому надежный лимит устанавливаем на 15000. F просто склеивает результаты двух независимых вызовов $G(n)$.

Задача 7: Условие

Задача

Алгоритм вычисления функций $F(n)$ и $G(n)$, где n — целое число, задан следующими соотношениями:

$$F(n) = 3 \cdot G(n-3) + 7;$$

$$G(n) = n + 2, \text{ если } n \leq 20;$$

$$G(n) = G(n-3) + 1, \text{ если } n > 20.$$

Чему равно значение выражения $F(37811)$?

Задача 7: Решение (Аналитически)

Анализ функции $G(n)$

Надо найти $F(37811) = 3 \cdot G(37808) + 7$.

Исследуем $G(37808)$. Спуск идет с шагом 3. Остаток от деления на 3:

$37808 \pmod 3 = 2$. Ближайшее число ≤ 20 с таким остатком — это 20.

Действительно, $(37808 - 20)/3 = 37788/3 = 12596$ полных шагов.

Значит: $G(37808) = G(20) + 12596$.

Так как $G(20) = 20 + 2 = 22$.

$G(37808) = 22 + 12596 = 12618$.

Подставляем в F :

$F(37811) = 3 \cdot 12618 + 7 = 37854 + 7 = 37861$.

Ответ

Ответ: 37861

Задача 7: Решение (Python)

```
import sys sys.setrecursionlimit(20000)
def G(n): if n <= 20: return n + 2
else: return G(n - 3) + 1
def F(n): return 3 * G(n - 3) + 7
print(F(37811))
```

Анализ кода:

Число 37811 разделенное на 3 дает около 12604 вызовов, так что устанавливаем лимит входа в 20000. Функция F вызывает $G(n-3)$, и *Python* берёт всю рекурсивную тяжесть на себя.

Задача 8: Условие

Задача

Алгоритм вычисления значения функции $F(n)$, где n — целое число, задан следующими соотношениями:

$$F(n) = n, \text{ если } n < 5,$$
$$F(n) = 2 \cdot n \cdot F(n - 4), \text{ если } n \geq 5.$$

Чему равно значение функции $(F(13766) - 9 \cdot F(13762))/F(13758)$?

Задача 8: Решение (Аналитически)

Раскрытие рекурсии

Шаг функции равен -4 . Раскроем бóльшие инстанции до $F(13758)$:

$$F(13766) = 2 \cdot 13766 \cdot F(13762) = 27532 \cdot F(13762).$$

$$\text{Значит } F(13766) - 9 \cdot F(13762) = 27532 \cdot F(13762) - 9 \cdot F(13762) = 27523 \cdot F(13762).$$

Теперь раскроем $F(13762)$:

$$F(13762) = 2 \cdot 13762 \cdot F(13758) = 27524 \cdot F(13758).$$

Подставляем в выражение:

$$(27523 \cdot 27524 \cdot F(13758)) / F(13758).$$

Сокращая $F(13758)$, получаем:

$$27523 \cdot 27524 = 757543052.$$

Ответ

Ответ: 757543052

Задача 8: Решение (Python)

```
import sys sys.setrecursionlimit(5000)
def F(n): if n < 5: return n else:
return 2 * n * F(n - 4)
ans = (F(13766) - 9 * F(13762)) //
F(13758) print(ans)
```

Анализ кода:

Значение 13766 велико, но при шаге -4 лимит потребуется в районе 3500, поэтому 5000 покрывает все нужды с запасом. Из-за умножений $F(n)$ возвращает число-гигант, поэтому чтобы не потерять точность, обязательно используем целочисленное деление (`//`). Вещественное деление (`/`) для огромных чисел приведёт к потере десятков младших цифр.

Задача 9: Условие

Задача

Алгоритм вычисления функций $F(n)$ и $G(n)$ (...):

$$F(n) = F(n-5) + 3219, \text{ если } n \geq 20;$$

$$F(n) = 8 \cdot (G(n - 9) - 34), \text{ если } n < 20;$$

$$G(n) = n/24 + 32, \text{ если } n \geq 250\,000;$$

$$G(n) = G(n+9) - 3, \text{ если } n < 250\,000.$$

Чему равно значение функции $F(925)$?

Задача 9: Решение (Аналитически)

Анализ функции

1) $F(925) = F(20 + 905)$. Шаг 5. Количество шагов до $n < 20$: $925 \rightarrow 15$ (остаток $925 \pmod{5} = 0 \rightarrow 15$ — ближайший < 20 , кратный 5). Шагов: $(925 - 15)/5 = 182$.

$$F(925) = F(15) + 182 \cdot 3219 = F(15) + 585858.$$

2) $F(15) = 8 \cdot (G(6) - 34)$. Ищем $G(6)$.

3) $G(6)$. База при $n \geq 250000$. Шаг G : $+9$, значение падает на 3.

Числа с таким же остатком: $6 \pmod{9} \equiv 6$.

Первое число ≥ 250000 с остатком 6: $\lceil 250000/9 \rceil \cdot 9 + ? \dots$ Проще: $249993 \equiv 0 \rightarrow 250000 \equiv 7 \rightarrow 250008 \equiv 6$.

Цель: $n = 250008$. Шагов: $(250008 - 6)/9 = 27778$.

$$G(6) = G(250008) - 3 \cdot 27778 = G(250008) - 83334.$$

$$4) G(250008) = 250008/24 + 32 = 10417 + 32 = 10449.$$

$$\text{Тогда } G(6) = 10449 - 83334 = -72885.$$

$$\text{Тогда } F(15) = 8 \cdot (-72885 - 34) = 8 \cdot (-72919) = -583352.$$

$$\text{Подставляем: } F(925) = -583352 + 585858 = 2506.$$

Ответ

Ответ: 2506

Задача 9: Решение (Python)

```
import sys sys.setrecursionlimit(30000)
def G(n): if n >= 250000: return n / 24
+ 32 else: return G(n + 9) - 3
def F(n): if n >= 20: return F(n - 5) +
3219 else: return 8 * (G(n - 9) - 34)
print(int(F(925)))
```

Анализ кода:

В ручном режиме легко ошибиться в поиске «ближайшего кратного», или перепутать минус с плюсом.

Python берёт всё на себя. Чтобы G(6) успело дойти до 250000, потребуется $\frac{250000}{9} \approx 27777$ рекурсивных вложений. Лимит 30000 идеален для этой задачи.

Задача 10: Условие

Задача

Алгоритм вычисления функций $F(n)$ и $G(n)$ (...):

$$F(n) = F(n-5) + 3480, \text{ если } n \geq 21;$$

$$F(n) = 10 \cdot (G(n-9) - 30), \text{ если } n < 21;$$

$$G(n) = n/20 + 33, \text{ если } n \geq 264\,685;$$

$$G(n) = G(n+9) - 2, \text{ если } n < 264\,685.$$

Чему равно значение функции $F(675)$?

Задача 10: Решение (Аналитически)

Анализ функции

1) $F(675)$. Спуск с шагом 5 до < 21 . $675 \pmod{5} = 0$. Число $20 \pmod{5} = 0$. Идем до $F(20)$.
Шагов: $(675 - 20)/5 = 131$. $F(675) = F(20) + 131 \cdot 3480 = F(20) + 455880$.

2) $F(20) = 10 \cdot (G(11) - 30)$.

3) $G(11)$. Шаг 9 вверх, значение падает на 2. База ≥ 264685 .

Остаток: $11 \pmod{9} = 2$. Ищем первое число ≥ 264685 с остатком 2.

$264685 \pmod{9} = 264681 + 4 \implies 264685 \equiv 4$.

Чтобы получить $\equiv 2$, прибавим 7: $264685 + 7 = 264692$.

Цель: 264692. Шагов: $(264692 - 11)/9 = 29409$.

$G(11) = G(264692) - 2 \cdot 29409 = G(264692) - 58818$.

4) $G(264692) = 264692/20 + 33 = 13234.6 + 33 = 13267.6$.

$G(11) = 13267.6 - 58818 = -45550.4$.

5) $F(20) = 10 \cdot (-45550.4 - 30) = 10 \cdot (-45580.4) = -455804$.

$F(675) = -455804 + 455880 = 76$.

Ответ

Ответ: 76

Задача 10: Решение (Python)

```
import sys
sys.setrecursionlimit(35000)
def G(n):
    if n >= 264685:
        return n / 20 + 33
    else:
        return G(n + 9) - 2
def F(n):
    if n >= 21:
        return F(n - 5) + 3480
    else:
        return 10 * (G(n - 9) - 30)
print(int(F(675)))
```

Анализ кода:

Ручной счёт в этой задаче ещё коварнее — возникают дробные числа (в ЕГЭ в ответах всегда целые, поэтому ошибка на десятую долю поведёт всё решение под откос). Вызовов будет ≈ 29409 , лимита в 35000 полностью хватит, а Python посчитает всё с идеальной математической точностью за доли секунды.